

Autonomous Robotic Systems

Finding Objects in a Scene

December 2024

Abstract—Behavioural robotics has been used to navigate a turtlebot within a simulated Gazebo environment to detect and visit coloured objects using image processing techniques. The resulting system is evaluated within various test environments and is found to explore the entire scene and identify these objects within an average of 14 minutes and 37 seconds.

I. INTRODUCTION

This project focuses on the development of a behavioural system, to explore a given world and identify all coloured objects present. Once a coloured object has been detected, the robot will beacon towards this and stop at an appropriate depth. The detection is announced through a command line message and by placing a visual marker on the RViz map.

While both members of the group contributed to the entirety of the project, the work was distributed to place focus on our strengths. This separated the project into two main areas: the robot's navigation system, and the development of the detection of objects. This report will focus on the latter section, discussing the identification of coloured objects within the scene, the control mechanisms to beacon towards these objects, and system used to prevent them from being re-visited.

II. METHODOLOGY

The robot follows a reactive behaviour using a closed loop control algorithm to respond to its environment. Vigorous testing was applied during each stage of the system design and development.

A. System Behaviour

The overarching behaviour of the system is represented within the Finite-State Machine shown in Figure 1. The lowest hierarchical behaviour is the goal-oriented exploration of the world, this is overwritten by any need response to stimuli based on information received from the integrated sensors.

There is a constant perceptual behaviour in which the robot performs a visual search of its current view of the world. If, during the exploration, an object of importance is detected, then the current goal is aborted, and navigation towards this object takes precedence. This stimulus response follows the idea of aggression discussed within Braitenberg's "Vehicles: Experiments in Synthetic Psychology", where there is a direct actuation link between motor behaviour and sensed data [1].

A secondary stimulus within the environment will otherwise initiate a fear response [1]. If an obstacle is detected ahead of the robot, at any time, aversive obstacle avoidance will take the highest priority to ensure that no collisions will occur.

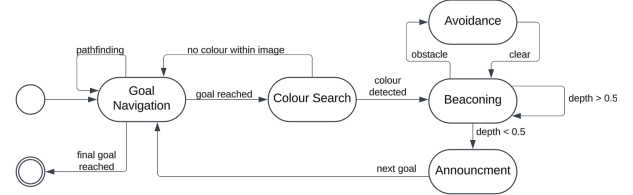


Fig. 1: Finite state machine of the overall behaviour.

B. Exploring the World

The exploration behaviour is implemented using the RViz pathfinding system within the pre-defined map. This allows for both memory-based and reactive navigation. Initially, localisation is performed to situate the robot within the map, this can be aided by publishing a 2D estimate of the initial position to the move_base topic using the RViz graphical interface [2]. Once successful, the first room waypoint is pushed as a goal to compute a path using Dijkstra's approach [3]. This is performed through a sense-plan-act cycle, to localise the robot's new position once the motion control has been executed [4]. This allows the path to be continually refined based upon the local collision map constructed through the captured information from the robot's sensors, to integrate the employment of aversive obstacle avoidance [5]. This is highly effective in avoiding collisions with both pre-defined static objects, such as walls and other permanent objects, and the additional obstacles within the scene [4].

The location of each waypoint is identified by publishing an estimate to the move_base/goal topic and echoing the resulting message to the terminal [2]. The published co-ordinate is retrieved and appended to the waypoints structure. Additional goals are introduced to force a desired heading before the next path is computed. This ensures the camera will capture a newly explored view to analyse.

A timeout value will abort a waypoint if it is found to be unreachable, this may be caused by the random positioning of objects within a goal co-ordinate. The initial solution considered a lookup of the local occupancy grid, to compare the current goal against all occupied map components [6]. This would use the robot's sensor data to make an informed decision as to whether the goal would be reachable. However, it was ultimately decided that the simplified approach would avoid any over-reliance on the RViz software.

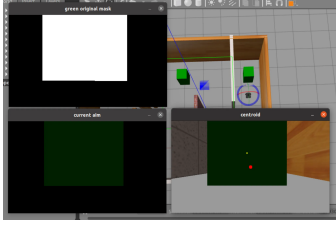


Fig. 2: An example result from development tests, showing the detection of a green object within the scene.

C. Detecting Objects

To identify the coloured objects within the scene, the robot's view must be accessed by subscribing directly to the RGB camera topic. This contains images generated from Gazebo that can be extracted within a callback function [7]. The incoming data stream must be converted into an OpenCV object to be prepared for processing. This can be achieved using the instantiated `cv_bridge` package provided within the OpenCV library [7] [8].

The image must then be converted into the HSV colour space, since the initial RGB values have a large dependence on the overall brightness of the scene. This would make it difficult to define a colour range that generalises across lighting conditions. The transformed HSV image separates these components into hue, saturation, and value, allowing changes in the brightness of the scene can be more easily encountered for [7].

When initialising the system, the lower and upper bounds are defined for each colour. To reliably detect the red coloured objects, two separate ranges are needed to wrap around the red regions within colour space. These were defined as between $[0, 230, 10]$ to $[9, 255, 255]$, and $[173, 230, 10]$ to $[180, 255, 255]$. The most efficient range for the green hues was found to be between $[40, 200, 20]$ and $[100, 255, 255]$.

A binary mask can be created over the captured image, containing positive values for the pixels within the defined colour ranges. This uses the included OpenCV function 'in-Range' [9], following the steps given within by Quigley et al.'s 'Programming Robots with ROS' [7].

As demonstrated in Figure 2, the created mask is then applied to this image to display an abstracted view and focus on the detected object. The two red masks are combined with a bitwise operation to capture all red positive cases. As shown in Figure 3, the initial mask is able to detect the majority of pixels, while the second mask detects pixels of a darker hue. The combination produces a solid patch that allows for accurate boundary detection.

Stray pixels may be introduced from objects containing similar hues. While a significant tuning process reduces this, the noise can be completely removed by filtering all minimal sized colour patches. This process makes use of the `findContours` function to separate the boundary of each coloured patch and calculate corresponding areas [10]. Increasing the permitted area removes larger patches that would incorrectly indicate the

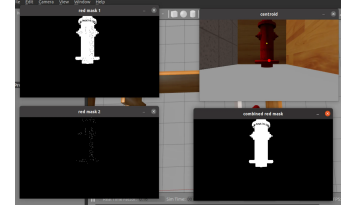


Fig. 3: An example comparing the output from each red mask and the combined mask that wraps around the colour space.

presence of an object. For example, the red sections within the cans shown in Figure 4. Following further evaluation, this limit was increased to a value of 250 to completely ignore these.

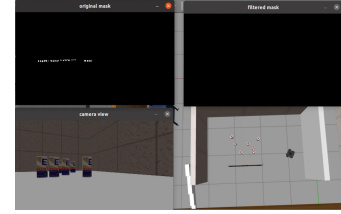


Fig. 4: The red pixels within the cans that are erroneously detected before the image is filtered.

D. Reactive Controller

When navigating the robot towards the detected object, beaconing is performed within a feedback loop. This has been developed based on the tutorial given by Quigley et al [7]. The centre of this object is calculated and indicated to the user by the yellow circle drawn over the image, as shown in Figure 2. The corrective error to adjust the robot's twist value is calculated as the scaled distance between this and the centre of the camera's view, indicated by the red circle [7].

As the robot moves towards the object, the published twist command must be continuously adjusted based on this error, to progress movement towards the acting beacon [7]. These changes must be fine to avoid extensive offshoot, otherwise the robot will oscillate between values and will not reach an alignment. The twist command is proportional to the size of the error required, where larger errors can afford larger corrections without risk.

An additional feedback check monitors the distance to the object, and publishes an empty twist message once this has been reached. This value can be extracted from the image published to the depth camera topic [11]. An alternate approach was built from the horizon following discussed by Quigley et al. [7]. This restricts the view to only consider objects directly ahead of the robot, and compares this against the desired view: a mask with a majority of positive pixels. However, the use of the depth camera was more effective at standardising the final stopping distance.

E. Acknowledging Found Objects

Objects that have previously been visited by the robot will not be visited again, this exploits the markers visualised within

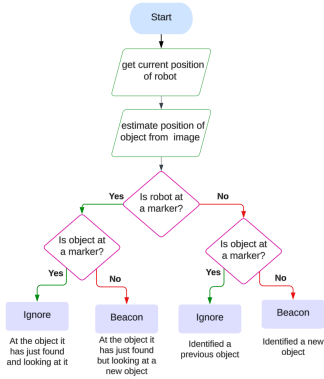


Fig. 5: A flowchart showing the different cases to consider before navigating towards an identified coloured object.

RViz [12]. Once an object is detected within the camera image, its position within the world space can be estimated relative to the robot's current co-ordinates. The euclidian distance can be calculated against each existing marker, if this is within a threshold value, there is an assumed match.

An issue was raised where the robot would become stuck at an object it had found, to overcome this there is an additional comparison against the robots position. Each resulting case is shown in Figure 5, and the executed behaviour is chosen based on assumptions made from these tests.

When first identifying an object, a reliable depth value cannot be extracted due to limitations of the depth camera. Therefore, the position cannot be reliably estimated, and the robot must assume that this is a new object and begin beaconing towards it. Once within acceptable range, the process can be completed and the robot can act accordingly.

Occasionally, this will cause an error within the system if the robot does not turn away from the object. If it remains within the camera's vision, it will again move into detectable range to re-perform this check and get trapped oscillating between these points.

III. EVALUATION

The detection of important objects has been evaluated by testing the performance accuracy within various world layouts. After completion of the development tests, the first round of evaluation focused on the detection of multiple objects within the camera image.

A. Image Processing

For the case where both objects are of the same colour, the robot performs as expected: placing its focus on the closest object and beaconing towards this. An example of this behaviour is given in Figure 6. The closest object is found by sorting the array of coloured boundaries based on their calculated areas [10]. The head of this can be indexed to find the largest object in the image. Once this object has been reached, the robot will resume navigation towards the previous waypoint by re-joining the original path. This will have a high

likelihood of revisiting the position where both objects were seen, and this time will select the second object.

When two objects are identified at the same distance, they will have similar areas within the created mask and the order will be chosen by the mapping function.

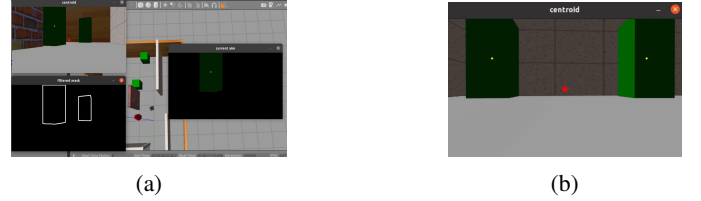


Fig. 6: The closest object is isolated within the robot's observations (a) and the computed centre of two objects at equal distance (b).

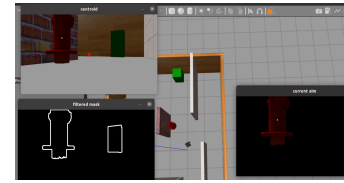


Fig. 7: Ideal behaviour focusing on the closest object with no prioritisation of colour, after adjustments have been made.

The system behaves inefficiently when two objects are seen of differing colours. Currently, green objects take priority, independent of their location to the robot. This causes significant backtracking and has redundant usage of resources. An efficient behaviour would allow the robot to visit the largest object of all colours, by combining the masks and applying this once onto the camera's image. An example of the resulting behaviour is given in Figure 7, however additional work is needed to adapt the announcements and markers to determine the colour of the object found and produce a correct label.

B. World Test Cases

The overall behaviour is analysed under differing world scenarios. These have varying levels of difficulty based on placement of objects, and differing parameters such as the number of obstacles to detect and the amount of additional objects to avoid collisions with. Throughout the experiment, performance metrics focus on the overall speed and success of the system; comparing both the number of objects correctly identified and the number of erroneous detections. The final results are presented in Table I.

TABLE I: Evaluation Results on Test Worlds

World	Time	Correct	Erroneous
Empty	13:37	0	0
Obstacles on Path	13:53	0	0
Sparse Objects	14:10	3	0
Objects with Obstacles	15:34	2	0
Dense Objects	15:52	2	0

1) *Empty World*: A benchmark time is calculated within an empty world, measuring the efficiency of the scene exploration and the impact object search has on this to highlight the weaknesses within the project. The time is measured once the robot has become oriented within the map and the first goal is successfully pushed.

Performance has been found to be greatly delayed when publishing goals to the move_base topic. Our group has encountered many issues with the RViz path finding being unpredictable, often resulting in the dropping of goals which can cause the robot to halt or result in unexpected movement off of the path.

2) *Small Obstacles*: The first evaluation is against an otherwise empty map with several small obstacles situated along the robot's path. Since the clutter 'cans' are successfully ignored within the colour detection stages, obstacle avoidance is handled during the ROS path-finding process. The results are shown within Figure 8.



Fig. 8: An empty world with a few small obstacles along the path to the final waypoint (a), and the updated local map (b).

3) *Sparse Objects*: The next case has introduced two green and one red object into the scene, positioned along the initial goal paths. As shown within Figure 9, the robot is able to identify all three objects and successfully navigate towards them without needing to detour - thus having a small impact on exploration time. Corresponding markers are posted to RViz once the object is reached.



Fig. 9: A world with few objects to identify along the robot's path (a), and the resulting published markers (b).

4) *Objects with Obstacles*: This world has two red objects placed within locations that are more difficult to reach. The first is situated against the brick wall, which will evaluate the removal of noise. While the second is placed off route with various small obstacles along the most obvious path. The test world and resulting map are given within Figure 10.

The robot performed well within this scenario, successfully locating the red objects by filtering the excess noise. However,

time efficiency has been greatly impacted due to the frequent path adjustments required to avoid the high concentration of small obstacles. The produced map shows the updated obstacle map used to visualise the estimated position of obstacles. This contains those detected before the initial localisation had been completed and therefore estimated to be in the wrong position. This has a small impact on the paths computed and is insignificant to the overall performance.



Fig. 10: A world with two red objects at difficult locations (a), and the resulting published markers (b).

5) *Dense Objects*: Two green objects have been placed close together to test the accuracy of determining whether an object has been visited before. These objects have been placed on either side of the brick wall to add additional verification on the filtering of green noise.

The overall time for this scenario was impacted greatly as there was a significant detour from the placed waypoints. While the system was able to identify both objects; as shown in Figure 11, one of the markers has not been placed with high accuracy, indicating there may be inconsistencies with performance. This may be due to the markers being placed at the robot's position at the time of the process being called, rather than storing the position and passing the value into the process. Due to the asynchronous nature of the project, there may be a delay between function calls allowing the robot to move further away from the object before the marker is placed.



Fig. 11: A world with two nearby green objects (a), and the resulting map (b).

IV. PERSONAL REFLECTIONS

From these evaluation tests, weakness have been identified within the system.

A. Finding Objects

A large inefficiency is caused by the green objects taking priority over the red ones. This is due to time constraints, which caused the last development cycle of object detection to be removed.

The system also requires an improved method to remember the position of objects that have been identified within the image but not yet visited. Currently, we rely on the robot revisiting its previous goal and viewing the objects again. One approach considered calculates the estimated position of each object and publishes them as goals. An alternate more naïve approach would retrace its steps to revisit the position where both objects were detected, then navigate towards the other. This must consider the accumulated error in the actuators to ensure the position is correctly revisited. The current beaconing method may progress the robot through the world, if there are no other objects detected. However, the robot must still revisit the previous waypoint. For most cases this is beneficial as this allows the robot to return to its previous position to search that area for more objects, there are scenarios where this can cause redundant map exploitation. Additional development of this system would explore the benefits of returning to the abandoned goal, and alternate navigation approaches.

B. Avoiding Blue Tiles

Avoidance of the blue tiles in the scene has not been implemented due to time constraints, however, our group has considered how this could be included.

As part of the mapping process, a cost-map is created to assign each node a value based on how it lends towards the pathfinding. Each goal-oriented path is computed based on these [6]. Therefore, in order to completely avoid the blue tiles, the corresponding nodes in the cost-map must be updated to a value of 254 so that these positions will not be considered when generating the path. Since these blue tiles can be situated at any position within the world, the local cost-map must be updated dynamically based on the information taken from the sensors.

These blue tiles must also be avoided when navigating towards the coloured objects, which can be implemented by pushing the object positions as goals rather than beaconing with the P-controller. The local-pathfinding will consider all of the information provided, including the known positions from the static map, the sensed obstacles within the scene, and the cost-map information.

A similar approach was initially considered, however there were a few issues when publishing goals to RViz causing the robot to not follow the created path and instead head directly towards the goal point. It is theorised that this is due to a publishing issue, due to time limits this was not successfully investigated and the use of RViz was minimised.

V. PEER EVALUATION

Group members: Natalie Robinson and Yap Pei Shan. Work has been distributed evenly, with the parts distributed as discussed within the introduction of this report.

REFERENCES

- [1] Valentino Braitenberg, *Vehicles: Experiments in Synthetic Psychology*, MIT Press, 1984.
- [2] ROS Wiki, "Using rviz with the navigation stack," .
- [3] ROS Wiki, "Sending goals to the navigation stack," .
- [4] Roland Siegwart, *Introduction to autonomous mobile robots.*, Intelligent robotics and autonomous agents. MIT Press, 2nd ed. / roland siegwart, illah r. nourbakhsh, and davide scaramuzza. edition, 2011.
- [5] ROS Wiki, "base_local_planner package summary," .
- [6] ROS Wiki, "costmap_2d package summary," .
- [7] Morgan Quigley, Brian Gerkey, and William D. Smart, *Programming Robots with ROS: A Practical Introduction to the Robot Operating System*, O'Reilly Media, Inc., 1st edition, 2015.
- [8] ROS Wiki, "Converting between ros images and opencv images (python)," .
- [9] OpenCV 3.4.20 dev Open Source Computer Vision, "Image processing - thresholding operations using inrange," .
- [10] Krutika Bapat LearnOpenCV, "Find the center of a blob (centroid) using opencv (c++/python)," .
- [11] ROS Wiki, "depth_image_proc package summary," .
- [12] HKU Robocon Tutorials, "Lab 6.3: Visualizations in ros with rviz," .